

William Jackson

Senior Software Engineer - Go

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjdev@outlook.com | linkedin.com/in/william-jackson-40036a3ba

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years of experience delivering resilient backend systems, data-intensive platforms, and production AI/ML services across the **telemetry, analytics, and intelligent automation industry** using **Go 1.18–1.23, Python 3.8–3.12, and TypeScript 4.x–5.x**. Strong expertise in **event-driven architectures, gRPC/REST platforms**, and reliability engineering, with consistent success reducing duplicate processing, stabilizing distributed workflows, and improving incident response through **OpenTelemetry**, structured logging, and actionable metrics. Known for modernizing brittle pipelines, hardening cloud services, and implementing safe rollout, retry, and backpressure strategies that keep high-volume systems observable, scalable, and operationally predictable.

SKILLS

- **Languages & Core** — **Go 1.18–1.23, Python 3.8–3.12, TypeScript 4.x–5.x**, JavaScript ES6+, SQL, Bash, Linux, concurrency patterns, memory profiling, performance tuning
- **Backend & APIs** — **gRPC, Protobuf, REST APIs**, middleware/interceptors, retries, timeouts, circuit breaking, rate limiting, backpressure, background workers, idempotency design, pagination, API versioning, WebSockets, service profiling
- **Frontend** — **React 17–18, Next.js 12–14**, Redux, HTML5, CSS3, Tailwind CSS, Webpack, Vite, component-driven UI patterns, internal tooling interfaces
- **Data, Messaging & Search** — **PostgreSQL**, MySQL, SQL Server, Redis, MongoDB, DynamoDB, **AWS SQS/SNS, Kafka**, RabbitMQ, **Elasticsearch, OpenSearch**, Redshift, BigQuery, Parquet, feature pipelines, dataset lineage, analytics event modeling
- **Cloud & DevOps** — **AWS**: EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM | **Azure**: AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights | **GCP**: GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery | Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI
- **Observability, Quality & Security** — **OpenTelemetry**, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch, structured logging, audit logging, Jest, Mocha, Chai, React Testing Library, Cypress, Playwright, API contract testing, **OAuth2, OpenID Connect, JWT**, Okta, Auth0, AWS Cognito, RBAC, ABAC, secrets handling, rate limiting

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viario Networks Inc. - Delaware, United States

- Architected a high-volume **Go** ingestion platform for bursty telemetry workloads, implementing bounded worker pools, replay-safe consumption, and backpressure controls that preserved downstream stability during producer spikes and partial infrastructure outages.
- Redesigned duplicate protection logic with **idempotency keys**, dedupe windows, and offset-aware processing, cutting double-counted event scenarios by more than **90%** during retry storms and broker rebalancing events.
- Modernized a fragile legacy **Python** scoring flow into a reproducible training and inference pipeline with immutable configuration, snapshot-based dataset handling, and controlled runtime inputs, which materially improved auditability and rollback confidence.
- Built a unified **gRPC** and **REST** access layer with shared timeout handling, request validation, authentication middleware, and standardized error envelopes, significantly reducing integration ambiguity for partner-facing consumers.
- Implemented centralized **OAuth2 / OpenID Connect / JWT** enforcement and policy-driven **RBAC/ABAC** checks, resolving inconsistent authorization behavior that had previously created edge-case access bugs across internal services.

- Introduced end-to-end **OpenTelemetry** tracing with correlation IDs and latency segmentation, allowing the team to isolate cross-service bottlenecks faster and improve mean time to detect production regressions by roughly **50%**.
- Designed a contract-governed feature storage model using **PostgreSQL** and **S3**, adding ownership metadata and validation rules that prevented silent schema drift from degrading downstream model behavior.
- Delivered safe canary rollouts for inference services on **Kubernetes** using health gates, traffic sampling, and comparative error monitoring, reducing risky production releases and improving rollout confidence by more than **40%**.
- Resolved recurring dashboard slowdowns by batching high-cardinality writes, reworking storage access patterns, and tuning **PostgreSQL** indexes, which kept operational dashboards responsive even under peak traffic windows.
- Implemented resilient reconciliation workers using **AWS SQS/SNS** with dead-letter handling and replay controls, eliminating many manual rerun tasks that had been required whenever upstream systems became unstable.
- Strengthened operational readiness with structured logging, field-level redaction, **CloudWatch** alerting, and **ELK/EFK** investigation workflows, giving on-call engineers faster and safer incident triage during customer-facing failures.
- Automated build, test, packaging, and deployment paths through **GitHub Actions** and **Jenkins**, creating repeatable release mechanics, safer rollback procedures, and more predictable delivery of backend services.

Software Engineer
Avantia, Inc. - Ohio, United States

October 2018 to January 2021

- Built production **Python** ETL pipelines that normalized inconsistent operational feeds into training-ready datasets, enforcing stable identifiers and timezone-safe timestamp handling to support reliable analytics and model preparation.
- Implemented deterministic preprocessing routines for missing values, category normalization, and feature encoding, reducing evaluation volatility by more than **30%** when source data distributions shifted unexpectedly.
- Created expectation-style validation layers with quarantine outputs and failure reporting, catching schema mismatches early and preventing bad records from silently contaminating downstream training and scoring workflows.
- Designed and tuned **PostgreSQL** schemas, indexes, and materialized extraction paths for nightly feature generation, resolving several SLA risks caused by heavy joins and poorly selective query patterns.
- Added incremental backfill tooling with watermark-based checkpoints and replay guards, enabling safe historical reprocessing without duplicate inserts or unnecessary stress on source APIs during recovery events.
- Developed a lightweight **Go** command-line utility that generated reproducible dataset slices for analysts, reducing manual SQL export work and improving consistency across experimentation and validation tasks.
- Instrumented internal services with correlated metrics and structured logs so a single customer's data path could be traced across ingestion, transformation, and scoring jobs during root-cause analysis.
- Built internal scoring APIs with clear **REST** error contracts, retries, timeouts, and circuit-breaker style safeguards, preventing upstream instability from cascading into broader service disruptions.
- Improved runtime efficiency by optimizing hot paths with **NumPy**, reducing I/O waste through **Parquet**, and freeing enough compute capacity to support additional experiments without proportional infrastructure cost growth.
- Introduced a lightweight model registry process with immutable artifacts, promotion metadata, and deployment references, ensuring released models always pointed to traceable inputs and reproducible build outputs.
- Led structured post-incident reviews for pipeline failures and recurring data defects, translating root causes into guardrails and steadily reducing repeated operational issues across the analytics delivery lifecycle.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Developed **Python** ETL workflows that consolidated multi-vendor event feeds, applying strict deduplication and timezone normalization so downstream reporting reflected consistent business reality across regions.

- Built internal **REST APIs** exposing curated datasets with documented pagination and stable response contracts, preventing fragile client integrations and reducing repeated support requests around large-result retrieval.
- Added regression protection around transformation logic using golden datasets and fixture-based checks, which preserved historical comparability and caught subtle behavioral regressions before release.
- Introduced **Docker**-based local development environments to eliminate machine-specific dependency issues, making defect reproduction faster and significantly reducing onboarding friction for new engineers.
- Optimized heavy reporting queries across **MySQL** and **PostgreSQL** through index design, join rewrites, and query-plan review, shrinking batch windows and improving report freshness for internal stakeholders.
- Implemented retry-aware background jobs with idempotent processing behavior, removing many manual rerun steps that had previously been needed after transient failures or partial external timeouts.
- Added service health checks, runtime diagnostics, and pragmatic **Linux** operational routines that surfaced failures earlier and reduced the duration of user-visible disruption during recurring incidents.
- Built reconciliation utilities that compared source counts with processed outputs, surfacing drift conditions early and preventing long-running discrepancies from quietly affecting stakeholder reporting.
- Improved maintainability by extracting shared parsing and transformation rules into reusable modules, reducing duplication and making enhancements safer across multiple ingest paths.
- Prepared systems for sustained growth by partitioning high-volume tables and introducing archival routines, keeping storage expansion predictable while preserving acceptable query performance over time.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Wrote **Python** and **SQL** utilities to clean, validate, and reconcile inconsistent customer datasets, adding defensive checks that stopped recurring export failures caused by format drift.
- Built small **Bash** automations for nightly pulls, comparisons, and delivery preparation, reducing spreadsheet-heavy manual work and creating more consistent audit-friendly outputs for the team.
- Implemented anomaly summaries and statistical checks for operational metrics, helping business users identify data quality problems before they influenced planning or customer-facing reporting.
- Standardized lightweight **Git** review practices and change checklists, improving collaboration quality and reducing avoidable merge-related defects during fast-moving delivery cycles.
- Improved scheduled job reliability through clearer exit-code handling, structured failure summaries, and practical troubleshooting guidance that made support work less guesswork-driven for engineers.
- Partnered with senior developers to document source systems, data ownership, and usage assumptions, reducing duplicate effort and clarifying definitions that had previously caused reporting confusion.
- Optimized a slow reporting workflow by redesigning table access patterns and adding appropriate indexing, making iterative analysis practical where runtime had previously blocked regular use.
- Supported safer incremental delivery by breaking larger changes into smaller deployable units, which reduced regression risk and made post-release troubleshooting materially easier.
- Contributed to production issue resolution on **Linux** hosts with pragmatic runbooks and recurring-failure notes, improving response consistency and helping the team recover faster from operational issues.

EDUCATION

Bachelor's degree in Computer Science
Harvard University

2010 to 2014

- Focused on strong fundamentals in **software engineering**, **databases**, and **systems**, building a foundation that translates into production-first delivery and disciplined operational practices.